Mini Tasks App — Full-Stack Assignment (≈4 hours)

Overview

Build a small **Tasks management web application** with:

- A backend API
- A SQL database
- A React frontend

The goal is to evaluate how you design and structure a small full-stack system.

Your solution should demonstrate:

- clean architecture
- maintainable code
- thoughtful API design
- a clear and intuitive UI

⚠️ **Important**

> Your code should be written as if this app might grow into a larger product.

That means we expect code that is:

- structured
- readable
- easy to extend
- logically organized

Database

A PostgreSQL database has already been provisioned.

You must **use the existing database exactly as provided**.

Do not create tables or modify the schema.

This ensures the reviewer can run your project immediately.

Database Connection

Use the following connection string:

postgresql://neondb_owner:npg_xzbV5H1rZJID@ep-misty-brook-a1bo3lnp-pooler.ap-southeast-1.aws.neon.tech/neondb?sslmode=require&channel_binding=require

Your backend must read this value from an environment variable.

Example `.env.example`

DATABASE_URL=<database connection string>
PORT=4000

Do not commit your `.env` file.

Existing Database Tables

The following tables already exist.

Users

column	type
id	integer (primary key)
email	varchar(255)
full_name	varchar(255)

Tasks

column	type
id	integer (primary key)
user_id	integer
title	varchar(255)
description	text
status	varchar
created_at	timestamp

Allowed values for status:

OPEN
IN_PROGRESS
DONE

Existing User

The database already contains a user:

id: 1
email: test@example.com
full_name: Test User

Your application can assume this user is always logged in.

You may simulate authentication however you prefer.

Backend

Build a backend service that allows the user to manage tasks.

The backend should:

- read from the provided database

- return JSON responses
- support task creation
- support task updates (CRUD)
- support listing tasks
- support searching and filtering tasks
- support pagination

Design your API in a way that would be reasonable for a production application.

Organize your code with clear separation of responsibilities.

Frontend

Build a simple web interface where the user can:

- view their tasks
- search tasks
- filter tasks by status
- paginate through tasks
- tasks CRUD UI
- change task status

The interface should include:

- a search input
- a status filter
- a task list or table

- a form to create tasks
- simple pagination controls

When updating a task's status, the interface should reflect the change clearly.

The UI does not need to be visually complex, but it should be **clean, intuitive, and usable**.

Handle loading states and errors appropriately.

Running the Application

The reviewer should only need to run the following commands.

Backend

cd backend
npm install
npm run dev

The backend should start on:

http://localhost:4000

Frontend

cd frontend
npm install
npm run dev

Open in browser:

http://localhost:5173

No database setup steps should be required.

Deliverables

Provide either:

- a Git repository link
  or
- a zip file

The project should contain:

backend/
frontend/
README.md

The README should briefly explain:

- how to run the backend
- how to run the frontend
- required environment variables

Evaluation Focus

We will evaluate submissions based on:

- code structure
- clarity of design
- correctness of functionality
- maintainability
- user experience